

testHub Automation Guide

The complete, interactive reference for the testhub-automation suite

UI

API

HYBRID

Module: automation/testhub-automation

Stack: Java 17 · Selenium 4.27 · TestNG 7.10 · RestAssured 5.5
ExtentReports 5.1 · Allure 2.29 · Log4j2 · Maven

Layout: src/common + src/api + src/ui + src/hybrid + src/resources

App under test: testHub - React SPA + API gateway + auth/core services

Generated: 12 Jun 2026

Contents

1. Overview	4
1.1 What this suite is	4
1.2 The app under test	4
1.3 The domain-split layout (and why)	5
1.4 The three design rules	5
2. Every class, explained	6
2.1 Configuration, constants & exceptions (src/common)	6
2.2 Driver management (src/common)	7
2.3 Enums (src/common)	9
2.4 Utilities (src/common)	10
2.5 Reporting & listeners (src/common)	14
2.6 Test data (src/common)	16
2.7 API layer (src/api)	16
2.8 UI layer - page objects (src/ui)	20
2.9 UI layer - tests (src/ui)	25
2.10 HYBRID layer (src/hybrid)	27
3. Execution flow - start to finish	30
3.1 From `mvn test` to TestNG	30
3.2 The shared test lifecycle, numbered	30
3.3 Flow diagram - a pure UI test	31
3.4 Flow diagram - a pure API test	32
3.5 Flow diagram - the hybrid test	33
3.6 Where exactly the three flows differ	33
4. How to run it	34
4.1 Prerequisites	34
4.2 Commands - profiles and overrides	34
4.3 Running from Eclipse	34
4.4 Where every artifact lands	34
4.5 Config reference - src/resources/config/config.properties	35
4.6 Known constraint: the register throttle (HTTP 429)	35
5. How to explain what I built	37

5.1 Explaining the UI framework	37
5.2 Explaining the API framework	37
5.3 Explaining the HYBRID layer	38

6. One-page cheat sheet	39
--------------------------------	-----------

1. Overview

1.1 What this suite is

testhub-automation is a UI + API test automation framework for **testHub**, a test-management web app. It is built on Java 17, Selenium 4 (WebDriver), TestNG, RestAssured, with dual reporting (ExtentReports and Allure), Log4j2 logging and Maven as the build/runner. It runs the same against a local stack, the live deployment or CI - only -D flags change.

The defining idea is a **clean split into three test layers**, color-coded throughout this guide:

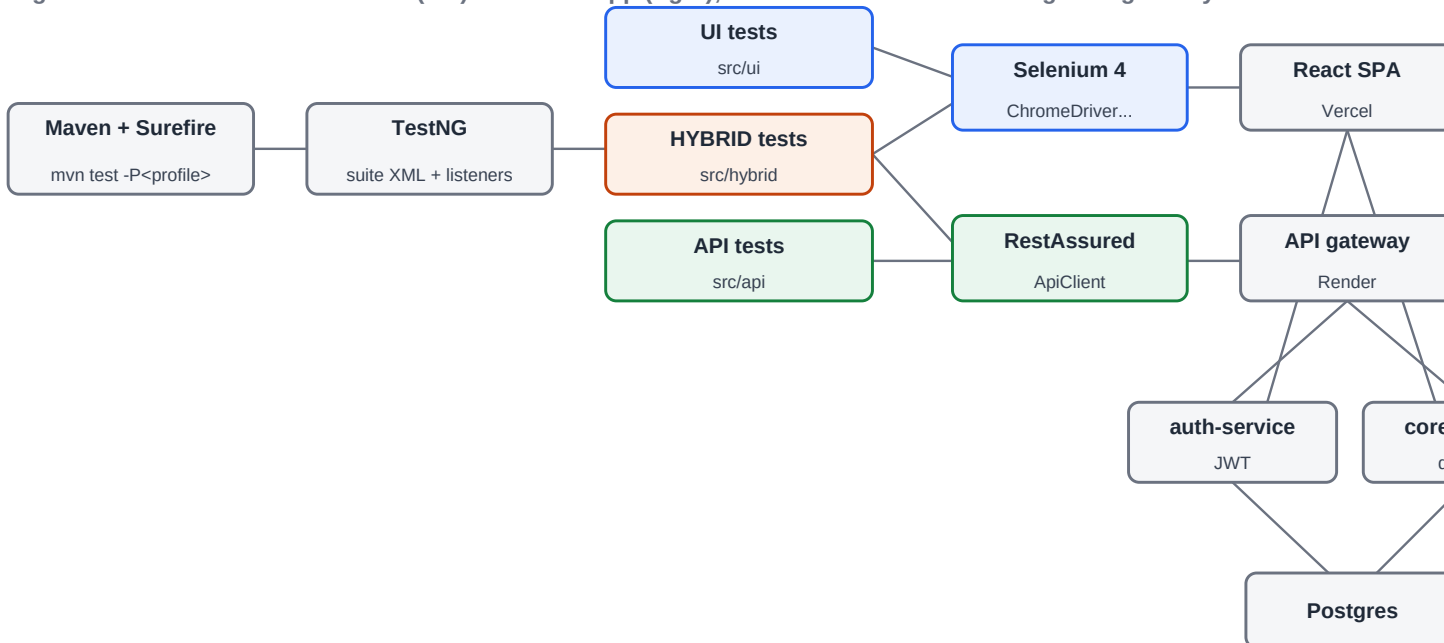
UI - browser only **API - RestAssured only** **HYBRID - API seeds, UI verifies**

Layer	Logs in via	Talks to	Lives in
UI	the real login/register forms (browser only)	the app like a human user	src/ui
API	POST /api/auth/login/register (RestAssured)	the gateway directly - no browser at all	src/api
HYBRID	the API, then injects tokens into the browser	API for setup speed, UI for verification	src/hybrid

1.2 The app under test

testHub is a small microservices product: a **React SPA** (Vercel) talks to an **API gateway** (Render), which fronts an **auth service** (register/login/JWT) and a **core service** (projects, suites, test cases, runs, results) backed by Postgres. It is hosted on a **free tier**: services fall asleep and cold-start in ~24 s+, and /api/auth/register is rate-limited (HTTP 429) under bursty traffic. Both facts shaped the framework - generous 40 s explicit waits, a wake-services flow, a one-shot retry, and an existing-account login strategy.

High-level architecture - framework (left) drives the app (right); SPA traffic also flows through the gateway



The SPA box is what UI tests drive with Selenium; the gateway box is what the API layer calls with RestAssured. Hybrid tests use both paths in one test.

1.3 The domain-split layout (and why)

Instead of Maven's classic `src/main/java` + `src/test/java`, the project is split **by domain**. Four source roots are registered with the build-helper-maven-plugin, so packages stay normal (`com.testhub.*`) while the folders give instant orientation: everything UI is under `src/ui`, everything API under `src/api`, shared plumbing under `src/common`, and the API-assisted tests are isolated - visibly - in `src/hybrid`.



Boxes are clickable - they jump to the matching class page in chapter 2.

1.4 The three design rules

#	Rule	Enforced how
1	UI tests log in ONLY through the real browser forms. No token shortcuts in <code>src/ui</code> .	<code>BaseTest</code> offers only <code>signUpFreshUser()</code> , <code>loginViaUi()</code> , <code>signInTester()</code> , <code>loginAsAdmin()</code> - all drive <code>LoginPage/RegisterPage</code> .
2	The API layer is independent. Nothing under <code>src/api</code> imports a page object or touches <code>WebDriver</code> .	<code>ApiClient</code> + <code>models</code> + API tests depend only on <code>RestAssured</code> , <code>Jackson</code> and <code>src/common</code> .
3	Exactly the hybrid tests bridge the two. If a test mixes API and browser, it lives in <code>src/hybrid</code> - nowhere else.	The <code>localStorage</code> token bridge exists in one place: <code>HybridBaseTest.injectApiSession()</code> .

2. Every class, explained

One entry per class - what it does, its key public methods, who calls it and what it depends on. Underlined names are internal links. Entries are grouped by module; the side color of each header table shows the layer: slate = common, blue = UI, green = API, orange = HYBRID.

2.1 Configuration, constants & exceptions (src/common)

ConfigManager - COMMON

Layer / package	COMMON · com.testhub.config
File	src/common/java/com/testhub/config/ConfigManager.java
Extends / implements	final class (static only)
Used by	every layer - DriverFactory , WaitUtils , ApiClient , BaseTest , tests
Depends on	FrameworkException , config/config.properties on the classpath

The single source of truth for configuration. Every lookup resolves in priority order: **1)** JVM system property (-Dbrowser=firefox), **2)** OS environment variable (UPPER_SNAKE_CASE, e.g. BASE_URL), **3)** config/config.properties. That is why the same build runs locally, against the live deployment and in CI with no code edits.

Key method	What it does
get(key) / get(key, default)	Raw lookup with the 3-level override chain; throws FrameworkException if a required key is missing.
baseUrl() / apiBaseUrl()	App and gateway URLs (trailing slash stripped).
browser() / headless()	Browser selection used by DriverFactory.
explicitTimeout() / pageLoadTimeout() / pollingInterval()	Durations behind every wait in WaitUtils.
reuseUser() / testerEmail() / testerPassword()	Existing-account login strategy for UI tests.
adminEmail() / adminPassword()	Standing ADMIN account credentials.
reportType() / extentEnabled() / allureEnabled()	Reporting switch: extent allure both.
retryCount(), warmupEnabled(), screenshotOnFailure(), screenshotOnSuccess()	Retry, warm-up and screenshot policies.

ConfigManager.java - the 3-level resolution chain

```
public static String get(String key) {
    String sys = System.getProperty(key);
    if (sys != null && !sys.isBlank()) return sys.trim();
    String env = System.getenv(key.toUpperCase().replace('.', '_'));
    if (env != null && !env.isBlank()) return env.trim();
    String value = PROPERTIES.getProperty(key);
    if (value == null)
        throw new FrameworkException("Missing configuration key: " + key);
    return value.trim();
}
```

FrameworkConstants - COMMON

Layer / package	COMMON · com.testhub.constants
File	src/common/java/com/testhub/constants/FrameworkConstants.java
Extends / implements	final class (static only)
Used by	paths: ScreenshotUtils , JsonDataReader , ExcelDataReader · routes: pages and BaseTest

Compile-time constants in one place so magic strings never leak: filesystem paths (test-output/ with reports/screenshots/downloads subfolders, src/resources/testdata), app routes (/login, /register, /, /projects, /test-cases, /users) and the two localStorage keys (accessToken, refreshToken) used by the hybrid session bridge.

FrameworkException - COMMON

Layer / package	COMMON · com.testhub.exceptions
File	src/common/java/com/testhub/exceptions/FrameworkException.java
Extends / implements	extends RuntimeException
Used by	thrown by ConfigManager , DriverManager , ApiClient , JsonDataReader , FileDownloadUtils

The framework's single unchecked exception. Wrapping low-level checked exceptions in it keeps page objects and tests free of try/catch boilerplate, and failures carry one clear, framework-level message (missing config key, non-2xx API call, unreadable data file...).

2.2 Driver management (src/common)

DriverFactory - COMMON

Layer / package	COMMON · com.testhub.driver
File	src/common/java/com/testhub/driver/DriverFactory.java
Extends / implements	final class (static only)
Used by	BaseTest .setUp() - @BeforeMethod creates one browser per test
Depends on	ConfigManager , DriverManager , BrowserType

Creates and configures the WebDriver. No driver binaries are managed by hand - Selenium 4's built-in **Selenium Manager** downloads the matching driver automatically. Chrome/Edge/Firefox options force downloads into test-output/downloads, and headless runs get a forced 1920x1080 window because headless 'maximize' is unreliable and would drop below the app's 1024px sidebar breakpoint.

Key method	What it does
createDriver()	switch on BrowserType -> new ChromeDriver/EdgeDriver/FirefoxDriver with options; sets page-load timeout + window size; registers the driver with DriverManager.
chromeOptions / edgeOptions / firefoxOptions(headless)	Per-browser launch args + download preferences.
applyExtraArguments(...)	Adds comma-separated args from the browser.arguments config key.

DriverFactory.createDriver() - core

```
WebDriver driver = switch (browser) {  
    case CHROME -> new ChromeDriver(chromeOptions(headless));  
    case EDGE -> new EdgeDriver(edgeOptions(headless));  
    case FIREFOX -> new FirefoxDriver(firefoxOptions(headless));  
};  
configureTimeoutsAndWindow(driver);  
DriverManager.setDriver(driver);
```


DriverManager - COMMON

Layer / package	COMMON · com.testhub.driver
File	src/common/java/com/testhub/driver/DriverManager.java
Extends / implements	final class (static only)
Used by	BasePage , WaitUtils , JavaScriptUtils , ScreenshotUtils - anything that needs the current browser

A **ThreadLocal<WebDriver>** holder - the key to safe parallel execution. Every TestNG thread gets its own driver; page objects pull it from here instead of passing it around. `getDriver()` fails fast with a clear message if no driver exists on the current thread.

Key method	What it does
<code>getDriver()</code>	Returns this thread's driver or throws <code>FrameworkException</code> .
<code>setDriver(driver)</code>	Package-private - only <code>DriverFactory</code> registers drivers.
<code>hasDriver()</code>	Null-safe check (used before screenshots).
<code>quitDriver()</code>	<code>driver.quit()</code> then <code>ThreadLocal.remove()</code> to prevent leaks.

DriverManager.java - ThreadLocal isolation

```
private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();

public static WebDriver getDriver() {
    WebDriver driver = DRIVER.get();
    if (driver == null)
        throw new FrameworkException(
            "No WebDriver for the current thread. Was the driver created in @BeforeMethod?");
    return driver;
}
```

2.3 Enums (src/common)

BrowserType - COMMON

Layer / package	COMMON · com.testhub.enums
File	src/common/java/com/testhub/enums/BrowserType.java
Extends / implements	enum: CHROME, FIREFOX, EDGE
Used by	DriverFactory

Maps the browser config string to a typed value; `from(value)` throws a clear `FrameworkException` for unsupported names.

Priority - COMMON

Layer / package	COMMON · com.testhub.enums
File	src/common/java/com/testhub/enums/Priority.java
Extends / implements	enum: LOW, MEDIUM, HIGH, CRITICAL
Used by	ApiClient.createTestCase , SuiteDetailPage , DataProviders

Test-case priority values exactly as the testHub UI/API accepts them.

ResultStatus - COMMON

Layer / package	COMMON · com.testhub.enums
File	src/common/java/com/testhub/enums/ResultStatus.java
Extends / implements	enum: PASS, FAIL, SKIP, BLOCKED
Used by	ApiClient .markResult, RunDetailPage , DashboardPage

Per-test-case outcome recorded inside a run; also drives the dashboard result-breakdown filter.

RunStatus - COMMON

Layer / package	COMMON · com.testhub.enums
File	src/common/java/com/testhub/enums/RunStatus.java
Extends / implements	enum: IN_PROGRESS, COMPLETED, ABORTED
Used by	TestRunsPage , RunDetailPage

Run lifecycle status. Each value carries a human label() ('In Progress'...) matching the run-status dropdown text in the UI.

Role - COMMON

Layer / package	COMMON · com.testhub.enums
File	src/common/java/com/testhub/enums/Role.java
Extends / implements	enum: ADMIN, TESTER, VIEWER
Used by	role assertions in tests

User roles. App rule: the first account ever registered on a fresh database becomes ADMIN; every later self-registration is a TESTER.

2.4 Utilities (src/common)

WaitUtils - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/WaitUtils.java
Extends / implements	final class (static only)
Used by	every page object via BasePage ; pages directly for URL waits
Depends on	DriverManager , ConfigManager

All synchronisation lives here - **explicit waits only**. There are no implicit waits and no Thread.sleep anywhere in page code. Default timeout is 40 s (generous on purpose: the free-tier backend can cold-start ~24 s+), polling every 1 s.

Key method	What it does
waitForVisible(locator[, timeout])	visibilityOfElementLocated - the workhorse.
waitForClickable(locator element)	Used by every click in BasePage.
waitForInvisible / waitForPresence / waitForAllVisible	Negative + presence conditions.
waitForUrlContains / waitForUrlToBe	Route-change assertions after login/register.
isPresent(locator)	Instant no-wait existence check.
waitForPageReady()	FluentWait until document.readyState == 'complete' (SPA load).

JavaScriptUtils - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/JavaScriptUtils.java
Extends / implements	final class (static only)
Used by	BasePage click fallback; HybridBaseTest session bridge; LoginPage

Thin JavascriptExecutor wrapper for the few cases where a native interaction is flaky: JS click when an overlay intercepts, scrollIntoView, and - crucially for the hybrid layer - setLocalStorage/getLocalStorage used to inject API-obtained JWT tokens into the browser.

Key method	What it does
execute(script, args...)	Raw executeScript.
clickWithJs(element)	Fallback when ElementClickInterceptedException is thrown.
setLocalStorage(key, value) / getLocalStorage(key)	Token injection for the hybrid session bridge.
scrollIntoView / scrollToBottom / clearLocalStorage	Misc helpers.

AlertUtils - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/AlertUtils.java
Extends / implements	final class (static only)
Used by	ProjectsPage .deleteProject (and any window.confirm flow)

Native browser dialog handling. testHub deletes projects/users via window.confirm(), which Selenium exposes as an Alert - acceptAlert()/dismissAlert() wait for it, capture its text and answer it.

Key method	What it does
waitForAlert()	Explicit wait until alertIsPresent.
acceptAlert() / dismissAlert()	Answer the dialog, returning its text.
isAlertPresent()	Instant check via switchTo().alert().

ScreenshotUtils - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/ScreenshotUtils.java
Extends / implements	final class (static only)
Used by	BaseTest .tearDown captures; TestListener embeds into the report

Captures screenshots to disk (PNG in test-output/screenshots) and as Base64 for inline report embedding. Its special trick: **captureAndStore()** snapshots the browser into a per-thread slot at the END of @AfterMethod - BEFORE driver.quit() - because TestNG fires listeners after @AfterMethod, when the driver is already gone.

Key method	What it does
capture(testName)	PNG to disk, timestamped, sanitised filename.
captureBase64()	In-memory Base64 PNG.
captureAndStore() / getStored() / clearStored()	The handoff between tearDown and TestListener.

JsonDataReader - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/JsonDataReader.java
Extends / implements	final class (static only)
Used by	DataProviders
Depends on	Jackson ObjectMapper; reads src/resources/testdata

Deserialises JSON test-data files into typed POJO lists (e.g. invalid-logins.json into List<LoginData>) and converts any list into the Object[][] shape a TestNG @DataProvider must return - one row per test invocation.

Key method	What it does
read(file, type) / readList(file, elementType)	Whole-file or top-level-array deserialisation.
readMaps(file)	Untyped list of maps.
toDataProvider(rows)	List -> Object[][] for TestNG.

ExcelDataReader - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/ExcelDataReader.java
Extends / implements	final class (static only)
Used by	DataProviders .casesFromExcel (ready for an .xlsx drop-in)
Depends on	Apache POI (XSSFWorkbook)

Reads .xlsx sheets into a list of header->value row maps (first row = header) - the Excel arm of the data-driven trio (JSON / CSV / Excel). Uses POI's DataFormatter so numbers and dates come back exactly as displayed.

Key method	What it does
read(fileName, sheetName)	Sheet -> List<Map<String,String>>; clear error if the sheet is missing.
toDataProvider(rows)	Rows -> Object[][] for TestNG.

FileDownloadUtils - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/FileDownloadUtils.java
Extends / implements	final class (static only)
Used by	CSV/JSON export download tests

Asserting on real browser downloads: the browser is forced (by DriverFactory prefs) to download into test-output/downloads, then waitForDownload(ext) polls until a completed file with that extension appears - ignoring partial .crdownload/.part files.

Key method	What it does
downloadDir() / clearDownloads()	Managed download folder.
waitForDownload(extension)	Newest completed file or FrameworkException after the explicit timeout.

TestDataFactory - COMMON

Layer / package	COMMON · com.testhub.utils
File	src/common/java/com/testhub/utils/TestDataFactory.java
Extends / implements	final class (static only)
Used by	every test that creates records (UI, API and hybrid)
Depends on	DataFaker

Generates realistic, **guaranteed-unique** test data: every name/email embeds a timestamp + random suffix, so parallel threads and repeated runs never collide on the shared backend - the framework's self-cleaning-data strategy starts here.

Key method	What it does
uniqueEmail()	qa_<millis><rand>@testhub.io
fullName() / password()	Faker name; password always satisfies the 6-char minimum.
projectName() / suiteName() / testCaseTitle() / runName()	Readable + unique entity names.
sentence() / steps() / expected()	Filler content for descriptions and case steps.

TestDataFactory.java - uniqueness

```
private static String unique() {
    return System.currentTimeMillis() + "" + FAKER.number().numberBetween(100, 999);
}
public static String uniqueEmail() { return "qa_" + unique() + "@testhub.io"; }
public static String password()    { return "Pass@" + FAKER.number().numberBetween(10000, 99999); }
```

2.5 Reporting & listeners (src/common)

ExtentManager - COMMON

Layer / package	COMMON · com.testhub.reports
File	src/common/java/com/testhub/reports/ExtentManager.java
Extends / implements	final class (static only)
Used by	TestListener ; BaseTest call setRunUser

Owns the single ExtentReports instance plus a ThreadLocal ExtentTest so parallel tests log into their own node. Writes to a hidden working file during the run, then every flush() copies it to test-output/reports/Latest/Report-<timestamp>_<user>.html; anything already in Latest/ is archived at the START of the next run. Dark theme, custom CSS, failed tests auto-expand.

Key method	What it does
getInstance()	Lazy init: archive previous run, configure Spark reporter, fill the environment panel.
setRunUser(email)	Once the signed-in user is known: updates the panel + report filename (null-safe when Extent is disabled).
setTest / getTest / unload	Per-thread ExtentTest accessor.
flush()	Writes the working file and copies it to Latest/ under one stable name per run.

TestListener - COMMON

Layer / package	COMMON · com.testhub.listeners
File	src/common/java/com/testhub/listeners/TestListener.java
Extends / implements	implements ITestListener, ISuiteListener
Used by	registered via @Listeners on BaseTest and in every suite XML
Depends on	ExtentManager , ScreenshotUtils , ConfigManager

Bridges TestNG lifecycle events into ExtentReports: creates a report node per test (title, description, groups, parameters), logs PASS/FAIL/SKIP with duration, embeds the stored screenshot, and pretty-prints the stack trace on failure. When `report.type=allure` it skips all Extent work - Allure records through its own auto-registered TestNG listener.

Key method	What it does
<code>onStart(suite) / onFinish(suite)</code>	Init Extent (if enabled) / flush the report.
<code>onTestStart(result)</code>	Create the ExtentTest node; logs '>>> START'.
<code>onTestSuccess / onTestFailure / onTestSkipped</code>	Result + duration + reason + stack trace + screenshot.
<code>attachScreenshot(test, status)</code>	Pulls <code>ScreenshotUtils.getStored()</code> - captured before the driver quit.

RetryAnalyzer - COMMON

Layer / package	COMMON · com.testhub.listeners
File	src/common/java/com/testhub/listeners/RetryAnalyzer.java
Extends / implements	implements IRetryAnalyzer
Used by	attached to every @Test by RetryTransformer

Retries a failed test up to `retry.count` times (default 1). The free-tier backend can throw one transient cold-start failure on the first hit; a single retry turns that into a pass instead of a flaky red. Set `retry.count=0` to disable.

RetryAnalyzer.java

```
@Override
public boolean retry(ITestResult result) {
    if (attempts < maxRetry) {
        attempts++;
        log.warn("Retrying '{}' (attempt {}/{}) after failure",
            result.getName(), attempts, maxRetry);
        return true;
    }
    return false;
}
```

RetryTransformer - COMMON

Layer / package	COMMON · com.testhub.listeners
File	src/common/java/com/testhub/listeners/RetryTransformer.java
Extends / implements	implements IAnnotationTransformer
Used by	registered as a listener in every suite XML

Applies [RetryAnalyzer](#) to every @Test automatically at load time, so no test has to opt in with `retryAnalyzer=...` by hand. Skips tests that already set their own analyzer.

2.6 Test data (src/common)

DataProviders - COMMON

Layer / package	COMMON · com.testhub.tests.data
File	src/common/java/com/testhub/tests/data/DataProviders.java
Used by	LoginTest (invalidLogins) and any data-driven test
Depends on	JsonDataReader , ExcelDataReader

Central home for TestNG @DataProviders - the data-driven backbone. `invalidLogins` feeds `LoginData` rows from `invalid-logins.json`, `testCases` feeds `TestCaseData` from `testcases.json`, `priorities` yields all four `Priority` values, and `casesFromExcel` is the Excel-driven variant (`testcases.xlsx`).

LoginData - COMMON

Layer / package	COMMON · com.testhub.tests.data
File	src/common/java/com/testhub/tests/data/LoginData.java
Extends / implements	POJO (Jackson)
Used by	LoginTest . <code>shouldRejectInvalidLogins</code>

One row of negative-login data: scenario, email, password, expectedError. `toString()` returns the scenario name so each data-driven run reads clearly in reports ('Wrong password', 'Malformed email...').

TestCaseData - COMMON

Layer / package	COMMON · com.testhub.tests.data
File	src/common/java/com/testhub/tests/data/TestCaseData.java
Extends / implements	POJO (Jackson)
Used by	data-driven test-case creation

One row of test-case creation data from `testcases.json`: title, description, steps, expected, priority - plus `priorityEnum()` to convert the string into the `Priority` enum.

2.7 API layer (src/api)

ApiClient - API

Layer / package	API · com.testhub.api
File	src/api/java/com/testhub/api/ApiClient.java
Extends / implements	plain class - one instance per authenticated session
Used by	API tests; HybridBaseTest + both hybrid tests for seeding
Depends on	RestAssured, Jackson, ConfigManager , FrameworkException

The RestAssured-backed client for the testHub gateway and the heart of the independent API layer. An instance carries an optional bearer token (set automatically by register/login), so the same client serves pure API tests and fast hybrid data-seeding. Every call goes through `ensure2xx()`, which throws a `FrameworkException` containing the HTTP status + body for any non-2xx response - that is how a 429 throttle surfaces with a readable message.

Key method	What it does
<code>register(name, email, password)</code>	POST /api/auth/register -> AuthResponse; stores the access token.
<code>login(email, password)</code>	POST /api/auth/login -> AuthResponse; stores the access token.
<code>loginOrRegister(name, email, password)</code>	Login first; on failure registers instead (seeded users).
<code>createProject(name, desc) / deleteProject(id)</code>	POST /api/projects, DELETE /api/projects/{id}.
<code>createSuite(projectId, name, desc)</code>	POST /api/suites/project/{projectId}.
<code>createTestCase(suiteId, title, ..., priority)</code>	POST /api/testcases/suite/{suiteId}.
<code>createRun(projectId, name, caseIds) / markResult(runId, caseId, status)</code>	POST /api/runs/project/{id}, POST /api/runs/{id}/results.
<code>isHealthy() / allServicesHealthy()</code>	GET /health on the gateway / on all three services.
<code>warmDomainServices()</code>	Best-effort GETs that wake the core service after a cold start.

ApiClient.java - request base + fail-fast

```
private RequestSpecification base() {
    RequestSpecification spec = given().config(CONFIG)
        .baseUrl(baseUrl).contentType("application/json").accept("application/json");
    if (token != null && !token.isBlank())
        spec.header("Authorization", "Bearer " + token);
    return spec;
}

private static Response ensure2xx(Response response, String action) {
    int code = response.getStatusCode();
    if (code < 200 || code >= 300)
        throw new FrameworkException("API " + action + " failed (HTTP " + code + "): "
            + response.getBody().asString());
    return response;
}
```

AuthResponse - API

Layer / package	API · com.testhub.api.models
File	src/api/java/com/testhub/api/models/AuthResponse.java
Extends / implements	POJO - @JsonIgnoreProperties(ignoreUnknown = true)
Used by	ApiClient deserialisation

Fields: user (UserDto), accessToken, refreshToken. Response of POST /api/auth/login and /api/auth/register. The two tokens are exactly what the hybrid layer writes into localStorage. All models ignore unknown JSON properties, so backend additions never break deserialisation.

UserDto - API

Layer / package	API · com.testhub.api.models
File	src/api/java/com/testhub/api/models/UserDto.java
Extends / implements	POJO - @JsonIgnoreProperties(ignoreUnknown = true)
Used by	ApiClient deserialisation

Fields: id, name, email, role. The signed-in user; role is ADMIN/TESTER/VIEWER. All models ignore unknown JSON properties, so backend additions never break deserialisation.

ProjectDto - API

Layer / package	API · com.testhub.api.models
File	src/api/java/com/testhub/api/models/ProjectDto.java
Extends / implements	POJO - @JsonIgnoreProperties(ignoreUnknown = true)
Used by	ApiClient deserialisation

Fields: id, name, description, status. A project; status defaults to ACTIVE on creation. All models ignore unknown JSON properties, so backend additions never break deserialisation.

SuiteDto - API

Layer / package	API · com.testhub.api.models
File	src/api/java/com/testhub/api/models/SuiteDto.java
Extends / implements	POJO - @JsonIgnoreProperties(ignoreUnknown = true)
Used by	ApiClient deserialisation

Fields: id, name, description, projectId. A suite, linked to its parent project. All models ignore unknown JSON properties, so backend additions never break deserialisation.

TestCaseDto - API

Layer / package	API · com.testhub.api.models
File	src/api/java/com/testhub/api/models/TestCaseDto.java
Extends / implements	POJO - @JsonIgnoreProperties(ignoreUnknown = true)
Used by	ApiClient deserialisation

Fields: id, title, description, steps, expected, priority, suiteId. A test case, linked to its suite. All models ignore unknown JSON properties, so backend additions never break deserialisation.

RunDto - API

Layer / package	API · com.testhub.api.models
File	src/api/java/com/testhub/api/models/RunDto.java
Extends / implements	POJO - @JsonIgnoreProperties(ignoreUnknown = true)
Used by	ApiClient deserialisation

Fields: id, name, description, status, projectId, selectedCaseIds. A test run and the case ids included in it. All models ignore unknown JSON properties, so backend additions never break deserialisation.

ApiBaseTest - API

Layer / package	API · com.testhub.tests.api
File	src/api/java/com/testhub/tests/api/ApiBaseTest.java
Extends / implements	abstract class
Used by	extended by AuthApiTest and ProjectApiTest

Base for pure-API tests. Unlike the UI BaseTest it **never launches a browser** - its only lifecycle is a @BeforeClass warm-up that polls the gateway /health (up to 8 x 3 s) so the first real request does not eat a cold start. Provides api() returning a fresh unauthenticated [ApiClient](#).

AuthApiTest - API

Layer / package	API · com.testhub.tests.api
File	src/api/java/com/testhub/tests/api/AuthApiTest.java
Extends / implements	extends ApiBaseTest

Pure-API auth coverage, 4 tests: registration returns user + both tokens and role TESTER; a registered user can log in (email round-trips lowercased); wrong password fails with HTTP 401; duplicate registration is rejected with HTTP 400. Negative cases assert via Assert.expectThrows(FrameworkException.class, ...) on the ensure2xx failure message.

ProjectApiTest - API

Layer / package	API · com.testhub.tests.api
File	src/api/java/com/testhub/tests/api/ProjectApiTest.java
Extends / implements	extends ApiBaseTest

Direct API coverage of the domain graph - and deliberately the same seeding helps the hybrid tests rely on, so an API regression is caught here first. 3 tests: create project returns id + ACTIVE; the full project -> suite -> case graph builds correctly (ids link up, priority round-trips); a project can be deleted. @BeforeMethod registers a fresh user per test.

2.8 UI layer - page objects (src/ui)

BasePage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/BasePage.java
Extends / implements	abstract class
Used by	every page object and component extends it
Depends on	DriverManager , WaitUtils , JavaScriptUtils

The keyword layer: centralises low-level Selenium so concrete pages read as business intent, never raw findElement/click. Every interaction waits explicitly first. click() falls back to a JS click if an overlay intercepts. Includes the two shared locator builders that exploit the app's markup: fieldByLabel(label) (label -> following input/textarea/select) and buttonByText(text).

Key method	What it does
click(locator element)	waitForClickable -> click; JS-click fallback on interception.
type(locator, text)	waitForVisible -> clear -> sendKeys.
getText / getValue / getTexts / findAll	Read helpers.
selectByValue / selectByVisibleText	Native <select> handling.
isDisplayed / isPresent / waitForReady	State queries.
fieldByLabel(label) / buttonByText(text)	Reusable XPath builders for the app's form markup.

BasePage.java - resilient click

```
protected void click(By locator) {
    WebElement element = WaitUtils.waitForClickable(locator);
    try {
        element.click();
    } catch (ElementClickInterceptedException e) {
        // An overlay/animation got in the way - fall back to a JS click.
        JavaScriptUtils.clickWithJs(element);
    }
}
```

BaseAppPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/BaseAppPage.java
Extends / implements	abstract, extends BasePage
Used by	every authenticated page (Dashboard, Projects, detail pages)

Base for every page behind login. Exposes the app shell shared by all of them - navbar(), sidebar() and modal() - so individual pages never re-declare the chrome.

NavbarComponent - UI

Layer / package	UI · com.testhub.pages.components
File	src/ui/java/com/testhub/pages/components/NavbarComponent.java
Extends / implements	extends BasePage
Used by	via BaseAppPage.navbar()

Top bar: logout() (waits for the /login redirect), openHelp(), toggleTheme(), getRole() - reads the ADMIN/TESTER/VIEWER role chip, used by login tests to assert the signed-in role.

SidebarComponent - UI

Layer / package	UI · com.testhub.pages.components
File	src/ui/java/com/testhub/pages/components/SidebarComponent.java
Extends / implements	extends BasePage
Used by	via BaseAppPage.sidebar()

Left navigation rail. Links are located by their stable hrefs (a[href="/projects"] etc.) rather than the responsive, duplicated label text. goToDashboard/Projects/TestCases/Users plus visibility checks used for RBAC assertions (isUsersLinkVisible).

ModalComponent - UI

Layer / package	UI · com.testhub.pages.components
File	src/ui/java/com/testhub/pages/components/ModalComponent.java
Extends / implements	extends BasePage
Used by	create/edit flows on Projects, Suites, Test Cases, Runs

The shared create/edit dialog. Because only one modal is ever open, BasePage's label-scoped field lookups resolve unambiguously inside it. waitUntilOpen()/waitUntilClosed() bracket every modal interaction so tests never race the open/close animation.

LoginPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/LoginPage.java
Extends / implements	extends BasePage
Used by	LoginTest , BaseTest ; the wake-services hook is used by openLoginPage()

The sign-in screen. Locators prefer the form's data-testid hooks (login-email, login-password, login-submit, wake-services, wake-status) - they survive styling changes far better than CSS classes. Also owns the cold-start helper wakeServicesAndWait(): clicks 'Services asleep? Wake them' and polls the status for up to 90 s until all services report awake. isEmailRejectedByBrowser() detects native HTML5 validation (type="email") blocking the submit - in that case no app error banner can appear.

Key method	What it does
open()	GET base.url + /login, wait for the email input.
loginAs(email, password)	Fill + submit + wait for the dashboard route -> DashboardPage.
submitLogin(email, password)	Fill + submit without asserting the outcome (negative tests).
isErrorDisplayed() / getErrorMessage()	App error banner.
isEmailRejectedByBrowser()	checkValidity() on the email input via JS.
wakeServicesAndWait() / getWakeStatus()	Free-tier cold-start wake flow.
goToRegister()	'Create an account' link -> RegisterPage.

RegisterPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/RegisterPage.java
Extends / implements	extends BasePage
Used by	RegisterTest , BaseTest .signUpFreshUser, the E2E journey

The account-creation screen. Fields have no test ids, so they are located by their visible labels (Name / Email / Password / Confirm password) via BasePage.fieldByLabel. registerAndLogin() auto-confirms the password, submits and waits for the dashboard route - the UI-only way every fresh user in this suite is born.

Key method	What it does
open()	GET /register, wait for the Name field.
submit(name, email, password, confirm)	Fill all four fields + submit (no outcome assertion).
registerAndLogin(name, email, password)	Happy path -> DashboardPage.
isErrorDisplayed() / getErrorMessage()	Client-side validation banner.
goToLogin()	'Log in' link back to sign-in.

DashboardPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/DashboardPage.java
Extends / implements	extends BaseAppPage
Used by	login/register flows land here; DashboardTest ; both hybrid tests

The landing dashboard: KPI stat cards, result-breakdown filter, recent runs. `waitUntilLoaded()` waits for the 'Dashboard' heading with the longer page-load budget because the heading only renders once the stats fetch resolves (absorbing a cold core-service). `getStatValue(title)` reads a KPI card ('Projects', 'Runs', 'Pass Rate') by its eyebrow title's sibling value.

Key method	What it does
<code>open()</code> / <code>waitUntilLoaded()</code> / <code>isLoading()</code>	Navigation + load gates.
<code>getStatValue(title)</code>	KPI card value by card title.
<code>filterByResult(status)</code> / <code>isResultFilterPanelVisible(status)</code>	Result-breakdown legend filter.
<code>refresh()</code> / <code>hasRecentRuns()</code>	Misc.

ProjectsPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/ProjectsPage.java
Extends / implements	extends BaseAppPage
Used by	ProjectsTest , E2E, ApiLoginExistingUserTest (hybrid verification)

The projects grid + create-project modal. `createProject()` opens the modal, fills Name/Description by label, submits and waits for the new card to render. `deleteProject()` clicks the card's Delete, accepts the native window.confirm via [AlertUtils](#), then waits for the card to disappear. Cards are located by an XPath keyed on the project's (unique) name.

Key method	What it does
<code>open()</code> / <code>isLoading()</code>	GET /projects, wait for heading + page-ready.
<code>createProject(name, description)</code>	Full create flow through the modal.
<code>isProjectVisible(name)</code> / <code>getProjectStatus(name)</code>	Card presence + ACTIVE badge.
<code>openProject(name)</code>	Card click -> ProjectDetailPage.
<code>deleteProject(name)</code>	Delete + confirm + wait for removal.

ProjectDetailPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/ProjectDetailPage.java
Extends / implements	extends BaseAppPage
Used by	E2E journey, ProjectsTest

A project's suites view ('Test Suites' heading). `getProjectId()` parses the id from the URL - the E2E test uses it to open the runs page. `createSuite()` drives the modal; `openSuite(name)` descends into a suite; `goToRuns()` jumps to the project's runs.

Key method	What it does
<code>open(projectId) / waitUntilLoaded() / isLoaded()</code>	Navigation + load gates.
<code>getProjectId()</code>	Project id from the current URL.
<code>createSuite(name, description) / isSuiteVisible / openSuite / deleteSuite</code>	Suite CRUD.
<code>goToRuns()</code>	-> TestRunsPage.

SuiteDetailPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/SuiteDetailPage.java
Extends / implements	extends BaseAppPage
Used by	E2E journey

A suite's test-case list ('Test Cases' heading). `createTestCase()` fills title, description, steps, expected and the Priority select inside the modal; `getPriority(title)` reads a case's priority chip back for round-trip assertions.

Key method	What it does
<code>open(suiteId) / waitUntilLoaded() / isLoaded()</code>	Navigation + load gates.
<code>createTestCase(title, desc, steps, expected, priority)</code>	Full create flow.
<code>isTestCaseVisible(title) / getPriority(title) / getVisibleCaseCount()</code>	Reads.
<code>deleteTestCase(title)</code>	Delete flow.

TestRunsPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/TestRunsPage.java
Extends / implements	extends BaseAppPage
Used by	E2E journey

A project's runs ('Test Runs' heading) with the two-step create wizard: createRun() fills name/description, clicks 'Next: Choose Test Cases', ticks each case by title, then submits via the 'Create Run (n)' button. Run cards support status change (setRunStatus) and deletion.

Key method	What it does
open(projectId) / waitUntilLoaded() / isLoaded()	Navigation + load gates.
createRun(name, description, caseTitles)	Two-step wizard.
isRunVisible / openRun(name)	Card presence / -> RunDetailPage.
setRunStatus(runName, status) / getRunStatusBadge / deleteRun	Card-level actions.

RunDetailPage - UI

Layer / package	UI · com.testhub.pages
File	src/ui/java/com/testhub/pages/RunDetailPage.java
Extends / implements	extends BaseAppPage
Used by	E2E journey

Inside one run: mark each case PASS/FAIL/SKIP/BLOCKED (markResult), read the per-case status and the summary counters (getSummaryCount('PASS')), change the run's own status via the Run Status select, and add more cases to a running run through the Add Test Cases modal.

Key method	What it does
open(runId) / waitUntilLoaded() / isLoaded()	Navigation + load gates.
markResult(caseTitle, status) / getResultStatus(caseTitle)	The core upsert-result flow.
getSummaryCount(key)	PASS/FAIL/... counters.
setRunStatus(status) / getRunStatus()	Run lifecycle.
openAddCasesModal / addCaseByTitle / goBackToRuns	Add cases + navigation.

2.9 UI layer - tests (src/ui)

BaseTest - UI

Layer / package	UI · com.testhub.tests
File	src/ui/java/com/testhub/tests/BaseTest.java
Extends / implements	abstract class - @Listeners(TestListener.class)
Used by	every UI test class + (via HybridBaseTest) the hybrid tests
Depends on	DriverFactory , DriverManager , LoginPage , RegisterPage , ScreenshotUtils

Shared lifecycle for every UI test. @BeforeSuite creates the output folders; @BeforeMethod creates a fresh per-thread driver; @AfterMethod captures the screenshot BEFORE quitting (and attaches it to Allure while the test case is still open), then quits. Authentication is **browser-only**: signUpFreshUser() drives the register form, loginViaUi() the login form, and signInTester() picks between the standing tester account (reuse.user=true, default - no registration, never throttled) and a fresh sign-up. The first visit to the login page wakes cold-start services exactly once per suite (double-checked, parallel-safe).

Key method	What it does
prepareSuite() [BeforeSuite]	mkdir test-output/{screenshots,reports,downloads}.
setUp() [BeforeMethod]	DriverFactory.createDriver() - one browser per test, per thread.
tearDown(result) [AfterMethod]	ScreenshotUtils.captureAndStore() -> Allure attachment -> quitDriver().
openLoginPage() / openRegisterPage()	Navigation + one-time service wake-up.
signUpFreshUser()	Register-form sign-up -> DashboardPage.
loginViaUi(email, password)	Login-form sign-in -> DashboardPage.
signInTester()	reuse.user ? standing tester login : fresh sign-up.
loginAsAdmin()	Login form with admin.email/admin.password.

BaseTest.java - fresh vs existing user

```
protected DashboardPage signInTester() {
    if (ConfigManager.reuseUser()) {
        return loginViaUi(ConfigManager.testEmail(), ConfigManager.testPassword());
    }
    return signUpFreshUser();
}
```

LoginTest - UI

Layer / package	UI · com.testhub.tests.ui
File	src/ui/java/com/testhub/tests/ui/LoginTest.java
Extends / implements	extends BaseTest

Pure-UI authentication, 4 tests: **valid login** through the form (standing tester when reuse.user=true; otherwise register -> logout -> login, all UI); **invalid logins** data-driven from invalid-logins.json - server-rejected rows must show the app's error banner with the expected message, while the malformed-email row is correctly blocked client-side by the type="email" HTML5 validation (asserted via isEmailRejectedByBrowser()); **empty credentials** shows a validation error; the **'Create an account'** link routes to /register.

RegisterTest - UI

Layer / package	UI · com.testhub.tests.ui
File	src/ui/java/com/testhub/tests/ui/RegisterTest.java
Extends / implements	extends BaseTest

Pure-UI registration, 5 tests: happy-path sign-up lands signed-in on the dashboard; mismatched passwords blocked client-side ('do not match'); short password rejected ('at least 6'); missing name rejected ('name is required'); the 'Log in' link routes back to /login.

ProjectsTest - UI

Layer / package	UI · com.testhub.tests.ui
File	src/ui/java/com/testhub/tests/ui/ProjectsTest.java
Extends / implements	extends BaseTest

Pure-UI project CRUD, 3 tests, signed in per-method via signInTester(). Create: new project appears in the grid with status ACTIVE. Delete: the confirm dialog is accepted and the card disappears. Open: clicking the card lands on the project's Test Suites view. Unique names mean a standing account's existing data never collides.

DashboardTest - UI

Layer / package	UI · com.testhub.tests.ui
File	src/ui/java/com/testhub/tests/ui/DashboardTest.java
Extends / implements	extends BaseTest

Pure-UI dashboard check for a genuinely fresh account: signs up through the form (always - this test ignores reuse.user because it asserts exact zero counts) and verifies the KPI cards render with Projects = 0 and Runs = 0.

EndToEndWorkflowTest - UI

Layer / package	UI · com.testhub.tests.e2e
File	src/ui/java/com/testhub/tests/e2e/EndToEndWorkflowTest.java
Extends / implements	extends BaseTest

The canonical product journey, 100% through the UI - no API shortcuts: register -> create project -> create suite -> create CRITICAL test case -> create run containing it (two-step wizard) -> mark it PASS -> verify the run summary -> open the dashboard and assert Projects = 1, Runs = 1, Pass Rate = 100%. This one test exercises almost every page object in the suite.

2.10 HYBRID layer (src/hybrid)

HybridBaseTest - HYBRID

Layer / package	HYBRID · com.testhub.tests.hybrid
File	src/hybrid/java/com/testhub/tests/hybrid/HybridBaseTest.java
Extends / implements	abstract, extends BaseTest
Used by	both hybrid tests
Depends on	ApiClient , JavaScriptUtils , FrameworkConstants

Base for the hybrid tests: a `@BeforeClass` gateway warm-up plus **the one bridging step that defines this layer** - `injectApiSession(auth)` takes the tokens an API login returned, navigates the browser to the app origin (localStorage is origin-scoped), writes `accessToken/refreshToken` into `localStorage`, and reloads on the dashboard route. The SPA reads the tokens and is signed in without ever seeing the login form. This bridge exists in exactly one place.

HybridBaseTest.java - the API->browser session bridge

```
protected void injectApiSession(AuthResponse auth) {
    if (auth.user != null && auth.user.email != null)
        ExtentManager.setRunUser(auth.user.email);
    // Must be on the app origin before localStorage is writable.
    DriverManager.getDriver().get(ConfigManager.baseUrl() + FrameworkConstants.ROUTE_LOGIN);
    JavaScriptUtils.setLocalStorage(FrameworkConstants.LS_ACCESS_TOKEN, auth.accessToken);
    JavaScriptUtils.setLocalStorage(FrameworkConstants.LS_REFRESH_TOKEN, auth.refreshToken);
    DriverManager.getDriver().get(ConfigManager.baseUrl() + FrameworkConstants.ROUTE_DASHBOARD);
}
```

ApiLoginDashboardTest - HYBRID

Layer / package	HYBRID · com.testhub.tests.hybrid
File	src/hybrid/java/com/testhub/tests/hybrid/ApiLoginDashboardTest.java
Extends / implements	extends HybridBaseTest

Hybrid test #1 - **fresh user**. Registers a brand-new account over the API, seeds TWO projects over the API (milliseconds, zero clicks), bridges the session into the browser, then verifies **through the real UI** that the dashboard's Projects KPI reads exactly '2'. A fresh account is required because the assertion relies on an exact count. Note: registration is throttled on the live free tier (HTTP 429 under bursty runs) - see the throttle-free variant below.

ApiLoginExistingUserTest - HYBRID

Layer / package	HYBRID · com.testhub.tests.hybrid
File	src/hybrid/java/com/testhub/tests/hybrid/ApiLoginExistingUserTest.java
Extends / implements	extends HybridBaseTest

Hybrid test #2 - **existing user**, in the smoke group. Logs in over the API as the standing tester (tester.email/tester.password - no registration, so it never touches the throttled endpoint), seeds one uniquely-named project over the API, bridges the session, verifies in the Projects UI that the project card is visible and the role chip reads TESTER - then deletes the project over the API in a finally block, leaving the standing account clean.

ApiLoginExistingUserTest.java - core flow (trimmed)

```
AuthResponse auth = api.login(ConfigManager.testEmail(), ConfigManager.testPassword());
String projectName = TestDataFactory.projectName();
ProjectDto project = api.createProject(projectName, "seeded via API (existing user)");
try {
    injectApiSession(auth);
    ProjectsPage projects = new ProjectsPage().open();
    Assert.assertTrue(projects.isProjectVisible(projectName),
        "The API-seeded project should be visible in the Projects UI");
} finally {
    api.deleteProject(project.id); // self-clean
}
```

3. Execution flow - start to finish

3.1 From `mvn test` to TestNG

Everything starts on the command line. The Maven profile only chooses which TestNG suite XML Surefire runs; the XML then decides packages, groups, parallelism and listeners:

The entry point - profile selects the suite XML

mvn test -Pui	->	src/resources/suites/ui.xml	(parallel=classes, 3 threads)
mvn test -Papi	->	src/resources/suites/api.xml	(parallel=classes, 3 threads)
mvn test -Phybrid	->	src/resources/suites/hybrid.xml	(serial)
mvn test -Psmoke	->	src/resources/suites/smoke.xml	(smoke groups, 2 threads)
mvn test	->	src/resources/suites/regression.xml	(DEFAULT)
mvn test -Pfull	->	src/resources/suites/full.xml	(everything)

Each suite XML registers two listeners before any test runs: [TestListener](#) (reporting bridge) and [RetryTransformer](#) (attaches [RetryAnalyzer](#) to every `@Test`). Allure's own TestNG listener registers itself automatically from the `allure-testng` jar. Surefire also starts the JVM with the AspectJ weaver agent so Allure annotations work.

3.2 The shared test lifecycle, numbered

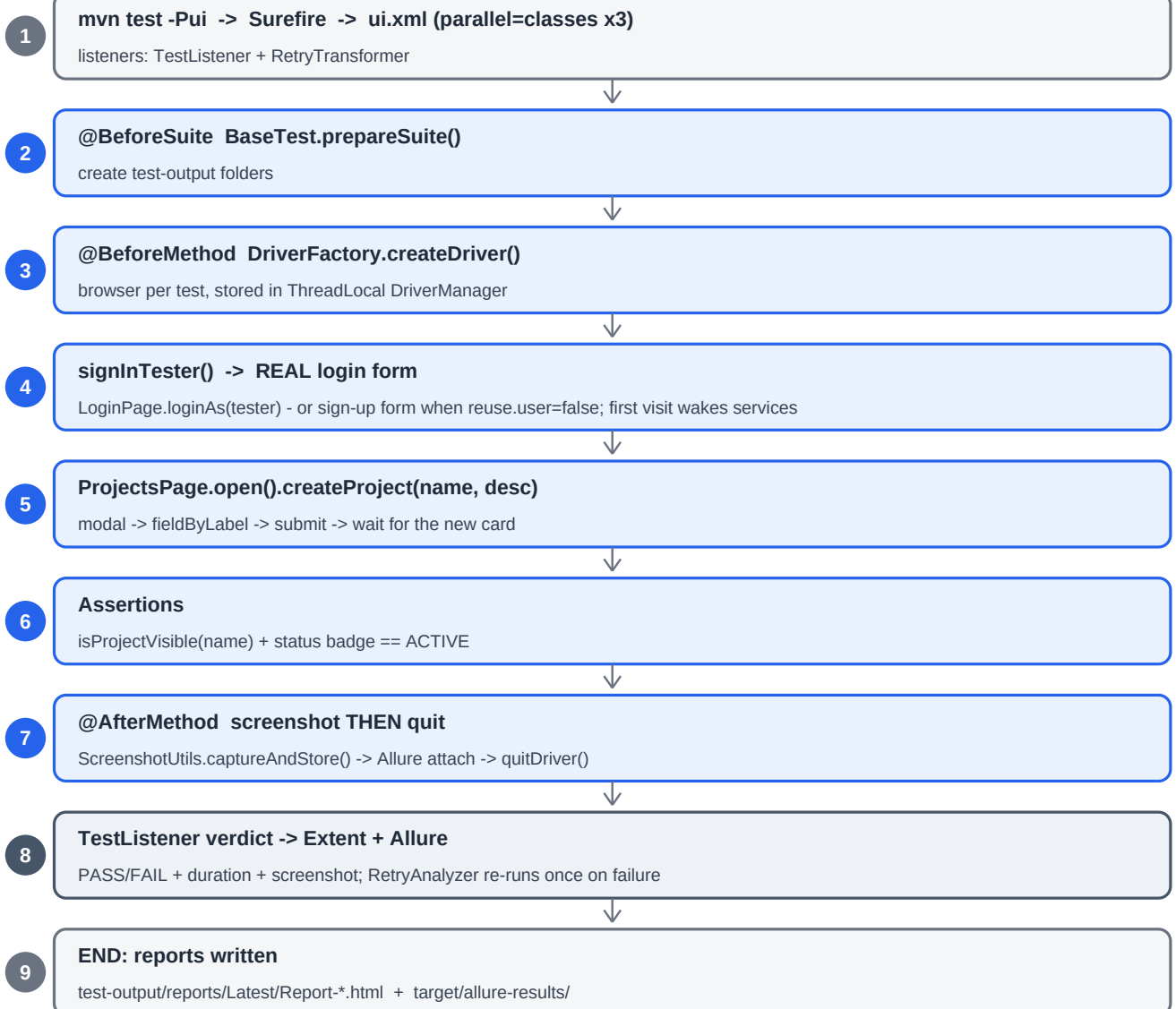
For every UI/hybrid test method, this is the exact order of events (API tests skip the browser steps - their diagram follows):

#	Phase	What happens (class responsible)
1	@BeforeSuite	BaseTest.prepareSuite() creates test-output/{screenshots, reports, downloads}.
2	Suite start	TestListener.onStart() initialises ExtentReports (archives the previous run's report).
3	@BeforeMethod	BaseTest.setUp() -> DriverFactory.createDriver(): browser launched, registered per-thread in DriverManager.
4	onTestStart	TestListener creates the ExtentTest node (title, description, groups, parameters).
5	Test body	Page objects -> BasePage actions -> WaitUtils explicit waits -> TestNG assertions. Sign-in is via the real forms (BaseTest helpers).
6	@AfterMethod	BaseTest.tearDown(): ScreenshotUtils.captureAndStore() (driver still alive!) -> Allure attachment if failed/skipped (or always when screenshot.on.success=true) -> DriverManager.quitDriver().
7	Listener verdict	TestListener.onTestSuccess/Failure/Skipped: PASS/FAIL/SKIP + duration + reason + stack trace + the stored screenshot into Extent.
8	Retry (only on failure)	RetryAnalyzer re-queues the test once (retry.count=1); the failed attempt is marked retried/skipped; steps 3-7 repeat.
9	Suite finish	TestListener.onFinish() -> ExtentManager.flush(): report copied to test-output/reports/Latest/Report-<timestamp>_<user>.html. Allure results are already in target/allure-results/.

3.3 Flow diagram - a pure UI test

Example: `ProjectsTest.shouldCreateProject()`. Click any box to jump to the class.

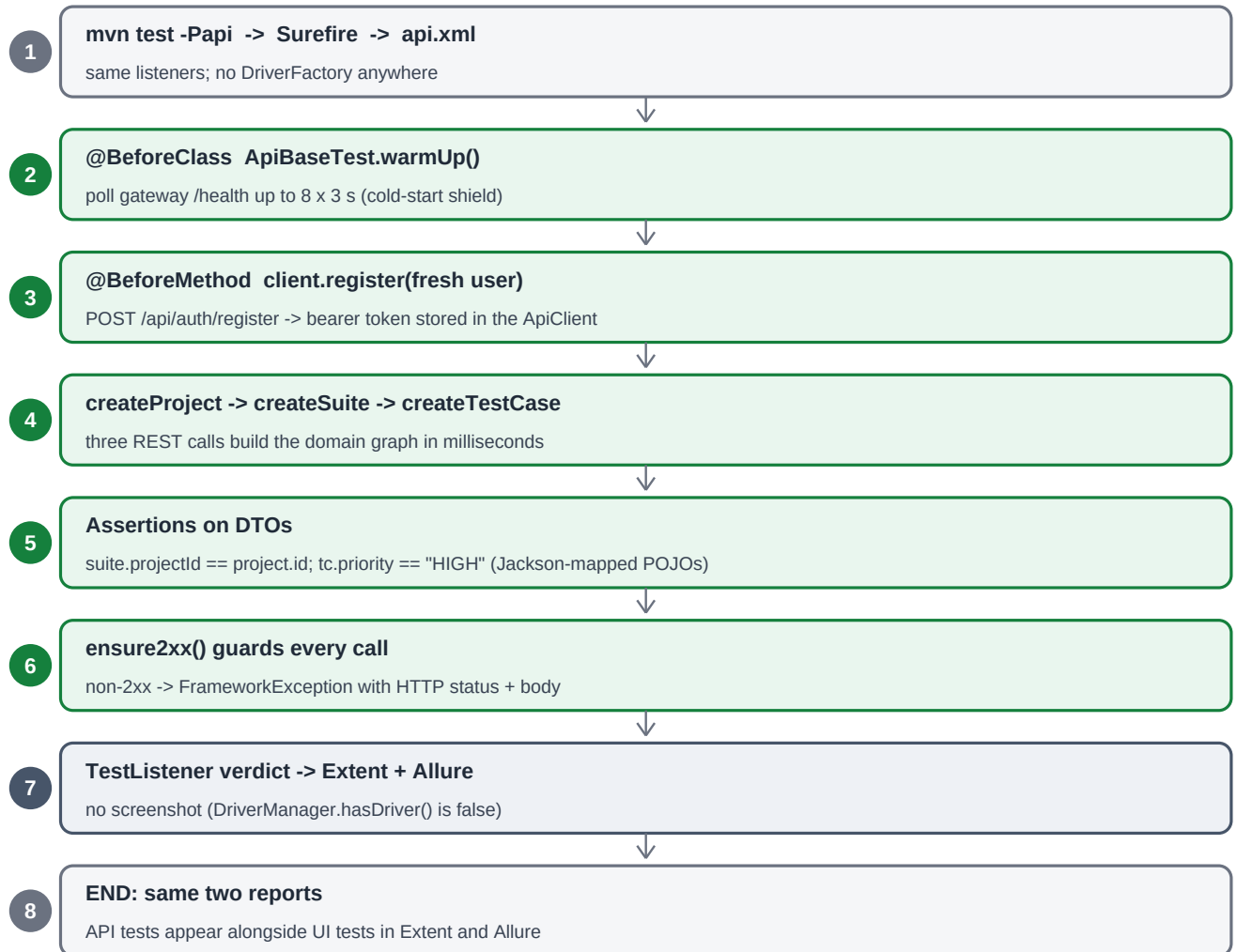
UI test - browser only, login through the form



3.4 Flow diagram - a pure API test

Example: `ProjectApiTest.buildsProjectSuiteCaseGraph()`. No browser exists at any point.

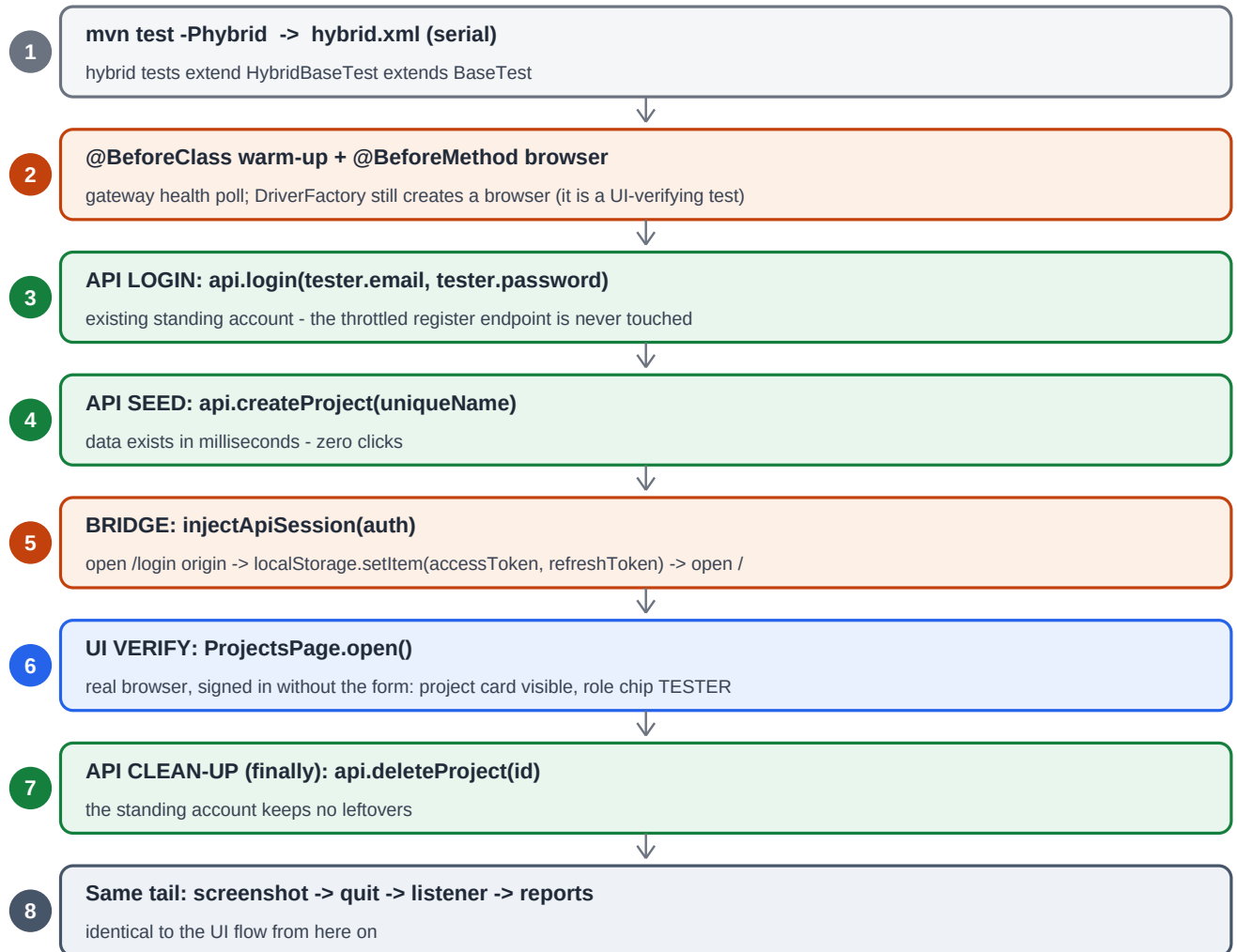
API test - RestAssured only, no browser



3.5 Flow diagram - the hybrid test

Example: ApiLoginExistingUserTest. Green steps are API, blue steps are UI - the orange step is the bridge that makes it hybrid.

HYBRID test - API for speed, UI for truth



3.6 Where exactly the three flows differ

Aspect	UI	API	HYBRID
Browser	yes - the whole test	never	yes - for verification
Login	real login/register form	POST /api/auth/* only	API login, tokens injected into localStorage
Data setup	clicks through the UI	REST calls	REST calls (seed) before UI checks
Base class	BaseTest	ApiBaseTest	HybridBaseTest (extends BaseTest)
Speed	slowest, most realistic	fastest	fast setup + realistic check
Folder	src/ui	src/api	src/hybrid

Why this matters: hybrid setup skips up to six screens of clicking before the assertion, which makes the test faster AND less flaky - while the thing being verified is still the real UI. And because the seeding endpoints are themselves covered by the pure API tests, a seeding bug cannot hide.

4. How to run it

4.1 Prerequisites

Requirement	Notes
JDK 17+	pom sets maven.compiler.release=17.
Maven 3.9+	mvn -version to verify.
Chrome / Edge / Firefox installed	No driver downloads - Selenium Manager fetches the matching driver automatically on first run.

4.2 Commands - profiles and overrides

Every run command
<pre>cd automation/testhub-automation mvn test # DEFAULT = regression.xml (UI smoke+regression, all API, hybrid) mvn test -Pui # pure-UI tests only mvn test -Papi # pure-API tests only (no browser) mvn test -Phybrid # the two API-assisted UI tests mvn test -Psmoke # fast smoke set across all layers mvn test -Pfull # everything # Common overrides (combine freely) mvn test -Pui -Dheadless=true # CI-style headless Chrome mvn test -Pui -Dbrowser=firefox # or edge mvn test -Papi -Dreport.type=allure # Allure only (extent allure both) mvn test -Pui -Dreuse.user=false # force fresh sign-ups instead of the standing tester # Point at a local stack instead of the live deployment mvn test -Pui -Dbase.url=http://localhost:5173 -Dapi.base.url=http://localhost:3000 # Any specific suite XML mvn test -DsuiteXmlFile=src/resources/suites/smoke.xml</pre>

4.3 Running from Eclipse

Step	How
Import	File > Import > Maven > Existing Maven Projects > select automation/testhub-automation. Project files (.classpath with the four source roots) are pre-configured.
Run a whole suite	Right-click src/resources/suites/<name>.xml > Run As > TestNG Suite (requires the 'TestNG for Eclipse' plugin from the Marketplace).
Run one class / method	Right-click a test class > Run As > TestNG Test. Note: suite-level listeners come from the XML, but reporting still works because TestListener is also on BaseTest via @Listeners; retry only applies via suite XML runs.
Pass -D flags	Run > Run Configurations > (your TestNG config) > Arguments > VM arguments, e.g. -Dheadless=true -Dreport.type=allure.

4.4 Where every artifact lands

Artifact	Location	How to view
Extent report	test-output/reports/Latest/Report-<timestamp>_<user>.html (previous runs auto-moved to reports/archive/)	Open the HTML file - fully self-contained, screenshots embedded

Artifact	Location	How to view
Allure results	target/allure-results/ (JSON + attachment PNGs)	mvn allure:serve (opens browser) or mvn allure:report -> target/site/allure-maven-plugin/
Failure screenshots (PNG)	test-output/screenshots/	Per-failure PNGs, timestamped
Run log	test-output/logs/automation.log	Log4j2 - every action, wait and verdict
Downloads under test	test-output/downloads/	Browser downloads are forced here by DriverFactory
Surefire XML/TXT	target/surefire-reports/	CI-friendly TestNG output

4.5 Config reference - src/resources/config/config.properties

Every key is overridable: **-Dkey=value** beats **ENV_VAR** beats the file (resolution implemented in [ConfigManager](#)).

Key	Default	What it changes
base.url	https://mytesthub.vercel.app	The SPA the browser opens. Local: http://localhost:5173
api.base.url	https://testhub-gateway.onrender.com	Gateway for the RestAssured layer. Local: http://localhost:3000
browser	chrome	chrome firefox edge
headless	false	Headless mode; window forced to 1920x1080 so the desktop sidebar stays visible
browser.arguments	(empty)	Extra comma-separated launch args
window.size	maximize	maximize or <w>x<h> (headed runs)
timeout.explicit	40 (s)	Every WaitUtils wait - generous to absorb ~24s cold starts
timeout.pageload	60 (s)	Driver page-load timeout + dashboard load gate
timeout.polling.millis	1000	Wait polling interval
admin.email / admin.password	admin14@gmail.com / (see file)	Standing ADMIN - used by loginAsAdmin()
tester.email / tester.password	testuser1@gmail.com / (see file)	Standing TESTER - used by signInTester() and the existing-user hybrid test
reuse.user	true	true = UI tests log in as the standing tester (no registration, no throttle); false = fresh sign-up per test
report.type	both	extent allure both - the reporting switch
screenshot.on.failure	true	Embed screenshot on failure
screenshot.on.success	true	Also screenshot passing tests (full visual audit)
retry.count	1	RetryAnalyzer attempts after a failure; 0 disables
warmup.enabled	true	Gateway health-poll warm-up before API suites; set false locally

4.6 Known constraint: the register throttle (HTTP 429)

The live deployment runs on free-tier infrastructure that **rate-limits** /api/auth/register. Bursty registrations - parallel threads each signing up a fresh user, or several full runs back-to-back - trip it, and the block then persists for a cooldown window (observed up to ~15 minutes). A throttled call fails fast as `FrameworkException: API register failed (HTTP 429): Too Many Requests` - deliberately surfaced, not hidden, because it is a hosting limit, not a product bug.

The strategy that avoids it (default config): **1)** reuse.user=true - UI tests sign in as the standing tester through the form, no registration at all; **2)** the smoke-grouped hybrid test logs in as the existing account over the API; **3)** tests that genuinely need a fresh account (DashboardTest zero-KPIs, the E2E journey, hybrid #1) stay few and are spread across runs; **4)** for unrestricted full-parallel runs, point at a local backend (-Dbase.url / -Dapi.base.url) where no throttle exists.

5. How to explain what I built

Three scripts - one per layer - each with a 30-second pitch, a deeper walkthrough, and the follow-up questions you are most likely to get, with strong answers.

5.1 Explaining the UI framework

30-second pitch:

"I built a Page Object Model framework in Java 17 with Selenium 4 and TestNG for a React test-management app. Pages expose business intent - createProject, markResult - while one base page centralises resilient interactions: every action goes through explicit waits, there is no Thread.sleep and no implicit wait anywhere, and clicks fall back to JavaScript if an overlay intercepts. It runs three browsers, parallel by class with a ThreadLocal driver, is data-driven from JSON, CSV and Excel, and reports to ExtentReports and Allure with failure screenshots - switchable by one config key."

Deeper walkthrough (2-3 min): Structure first: a **keyword layer** (BasePage: click/type/select with waits baked in), **components** for the app shell (navbar, sidebar, shared modal) so pages never re-declare chrome, and **thin page objects** on top. Locator strategy is deliberate: data-testid where the app provides hooks (login form), stable hrefs for navigation, and label-based XPath builders (fieldByLabel) exploiting the app's consistent form markup. Lifecycle: fresh browser per test method, screenshot captured before driver.quit() because TestNG fires listeners after @AfterMethod. Test data is generated unique per run (timestamp + random) so parallel tests on a shared backend can never collide - and the suite can sign in either as a standing account (default, avoids a free-tier register throttle) or with a fresh sign-up per test for max isolation, one config flag.

Likely follow-up	Strong answer
Why ThreadLocal for the driver?	TestNG runs classes on different threads (parallel=classes, 3 threads). A static driver would be shared and corrupted; ThreadLocal gives each thread its own isolated instance with zero locking. quitDriver() also removes the entry so threads do not leak drivers across tests.
How do you handle flakiness?	Three layers: (1) every interaction waits explicitly for the right condition - no sleeps, so no timing guesses; (2) BasePage click falls back to a JS click when an overlay intercepts; (3) a one-shot RetryAnalyzer absorbs genuine infrastructure flakes (cold starts), while real failures still fail. Flaky-by-design things, like the app's 90 s wake-up flow, have their own dedicated handling in LoginPage.
Why no implicit waits?	Implicit + explicit waits interact unpredictably (Selenium docs warn against mixing), and implicit waits hide real performance problems. Explicit conditions document exactly what each step needs - visible, clickable, URL changed - and fail with a precise message.
How is it data-driven?	TestNG @DataProviders fed by external files: JSON (Jackson into typed POJOs like LoginData), CSV, and Excel via POI. The invalid-login test runs once per JSON row, and each row names its scenario so reports read like a test matrix.
What was the hardest UI bug?	A 'malformed email' negative login failed because no app error banner appeared - the root cause was the input being type="email", so the browser's native HTML5 validation blocked submission before the app could react. The fix asserted the real behavior: client-side rejection via checkValidity() instead of expecting a server banner.

5.2 Explaining the API framework

30-second pitch:

"Alongside the UI suite I built an independent RestAssured layer against the app's gateway: a typed ApiClient covering auth, projects, suites, test cases, runs and results, with Jackson POJO models and a fail-fast guard that turns any non-2xx into a readable exception with the status and body. Pure API tests cover registration, login, negative auth and the whole domain graph - and the same client doubles as the data-seeding engine for hybrid UI tests."

Deeper walkthrough: The client is instance-based: register()/login() store the JWT on the instance, so an authenticated session is just an object you pass around - clean for parallel tests, no static token state. Models are

tolerant (@JsonIgnoreProperties) so backend additions never break deserialisation. The API tests deliberately exercise the exact endpoints the hybrid layer seeds with - if seeding breaks, the API suite fails first and points at the real layer. There is also an ops-aware touch: a health-poll warm-up before suites because the free-tier gateway cold-starts, and 4xx responses are surfaced verbatim - which is how we caught the register throttle as HTTP 429 rather than as a mystery timeout.

Likely follow-up	Strong answer
Why RestAssured?	Industry standard for Java API testing: readable given/when/then specs, first-class Jackson integration, easy header/auth handling. I centralised the base spec (baseUri, JSON content type, bearer header) so tests never repeat plumbing.
How do you assert failures?	ensure2xx throws FrameworkException carrying the HTTP code and response body; negative tests use Assert.expectThrows and check the message contains 401/400. One mechanism for transport errors everywhere.
Why are API tests browser-free?	Speed and signal: they run in milliseconds, in parallel, with no rendering noise. A gateway regression fails here in seconds instead of surfacing as a confusing UI timeout.

5.3 Explaining the HYBRID layer

30-second pitch:

"The hybrid tests are the bridge: authenticate and seed data over the API in milliseconds, inject the JWTs into the browser's localStorage, and then verify through the real UI. One hybrid test registers a fresh user and proves the dashboard KPIs reflect two API-seeded projects; the other logs in as an existing account - completely avoiding a free-tier register throttle - seeds a project, verifies it in the UI, and deletes it again so the account stays clean. They live in their own source folder, so anyone can see at a glance which tests are API-assisted."

Deeper walkthrough: The pattern is 'seed over the API, verify through the UI'. A pure UI test of 'dashboard reflects data' would click through six screens before the assertion - slow and fragile. Instead HybridBaseTest.injectApiSession() navigates to the app origin (localStorage is origin-scoped), writes accessToken/refreshToken - the same keys the SPA itself uses - and reloads; the app boots signed-in. The bridge is one protected method in one base class, so the 'UI tests never bypass the login form' rule stays auditable: if a test imports ApiClient and a page object, it must live in src/hybrid.

Likely follow-up	Strong answer
Why inject tokens instead of logging in through the UI?	Login-through-the-form is covered - thoroughly - by the UI suite. Repeating it in every other test adds ~10 s and a flake point per test while verifying nothing new. Token injection makes test setup orthogonal to the login feature, exactly like API seeding makes it orthogonal to the create-project modal. The login feature itself still has dedicated UI coverage.
Why a separate src/hybrid folder?	Discoverability and rule enforcement. The folder IS the documentation: pure UI suites stay honest (no hidden API shortcuts), and reviewers can spot a misplaced test by its imports alone.
Fresh vs existing user - why both hybrid tests?	They demonstrate the trade-off: exact-count assertions (Projects KPI = 2) need a virgin account, so test #1 registers fresh - but that endpoint is throttled on the free tier. Test #2 shows the production-friendly pattern: existing account, relative assertions (my uniquely-named project is visible), and API clean-up in finally so it is rerunnable forever.
Is localStorage injection safe/realistic?	It uses the app's own session mechanism - the same two keys the SPA writes after a real login - so the app behaves identically. It would not work for HttpOnly-cookie sessions; there I would reuse the API login response via Selenium CDP or a session-cookie injection instead. Knowing where the technique applies is part of the design.

6. One-page cheat sheet

Commands

```
cd automation/testhub-automation
mvn test -Pui | -Papi | -Phybrid | -Psmoke | -Pfull      (no -P = regression)
  -Dheadless=true   -Dbrowser=firefox|edge   -Dreport.type=extent|allure|both
  -Dreuse.user=false -Dbase.url=... -Dapi.base.url=... (local stack)
Reports: test-output/reports/Latest/*.html | mvn allure:serve
```

Folder map

```
src/common - config, driver, waits, utils,
              reports, listeners, data
src/api    - ApiClient + 6 DTOs + API tests
src/ui     - BasePage + 8 pages + 3 components
              + BaseTest + 5 UI test classes
src/hybrid - HybridBaseTest + 2 hybrid tests
src/resources - config.properties, 6 suite
              XMLs, testdata/
```

Golden rules

1. UI tests sign in via the real forms only.
2. src/api never imports UI code.
3. API + browser in one test? -> src/hybrid.
4. No sleeps - explicit waits only (40 s budget).
5. Test data is always unique (timestamp+random).
6. Screenshot BEFORE driver.quit().
7. reuse.user=true avoids the 429 register throttle.

	UI flow	API flow	HYBRID flow
1	browser starts	health warm-up	browser starts + warm-up
2	REAL form login (signInTester)	api.register/login (token in client)	API login (existing tester)
3	click through pages	REST calls build the graph	API seeds data
4	assert in the UI	assert on DTOs	inject tokens -> localStorage
5	screenshot -> quit	ensure2xx guards all calls	verify in the REAL UI
6	Extent + Allure	Extent + Allure	API clean-up -> reports

Layer colors used everywhere in this guide: **UI** · **API** · **HYBRID** · **common**